PixelFlow 技术汇报大纲

一、叙述主线

以客户端中的用户行为为起点，解释产品要求如何转化为技术设计。

两类行为贯穿汇报：

1. **连续浏览内容。** 用户滑动双列瀑布流、打开详情、返回列表，希望图片及时出现、内容连续、浏览位置稳定。因此介绍预加载、缓存、图片内存控制和页面状态保留。
2. **获取与整理信息。** 用户提出问题、查找资料、整理为自己的作品，希望系统理解任务、调用社区能力，并保留对内容的控制。因此介绍 Agent、Skill、创作流程和 MCP。

两条主线通过“AI 运行期间仍能继续使用社区”衔接。这样请求调度、流式展示与任务恢复都有明确的业务动机。

建议汇报题目：**PixelFlow：围绕浏览与创作的系统设计**。

内容约束

- 项目背景、基本功能和总体架构合计只讲约 15 秒，不设独立背景页、目录页或技术栈罗列页。
- 一个技术问题可以用多页展开；每页只推进一个主要论点。
- 命中率、耗时占比等数据来自真机 A/B 对照实验，仅陈述测量结果；不宣称未经实验支撑的绝对帧率、“零卡顿”“秒开”“绝不 OOM”等表述。
- **本文档为答辩大纲草稿：推荐部分（第 5、6 页）暂按“首页已接入个性化推荐”的假设撰写，实际闭环待实现**（方案见 `docs/android/推荐闭环实现方案-20261111.md`），落地后需回填该假设。

二、核查范围与选材结论

主线 A：连续浏览体验

用户需求	已找到的实现	汇报价值	边界
首页内容个性化	行为画像（曝光/点击/停留/点赞 → 30 天窗口规则聚合）+ 三个 Skill 规则推荐（profile_insight → candidate_recall → feed_rank）+ 首页个性化 Feed（**待接入，PPT 暂按已闭环假设**）	讲清行为→画像→推荐→首页的可解释闭环；推荐是有界确定性编排，非黑盒大模型	主题用标题关键词分类、非语义 候选池仅 50 条；画像有更新延迟 推荐是规则组合、非训练模型
滑到下一屏时尽早有图	根据滚动方向预加载相邻最多 3 项；500ms 触发间隔； 停止滚动时处理可视区； 详情入口有高优先级预加载调用	用真机 A/B 数据说明预加载价值（网络占比 50%→7.4%）与预解码取舍（不预解码 →12.5%+失败）	预加载对象主要是图片与视频封 不等于预下载完整视频； 不是训练得到的预测策略
列表底部内容及时衔接	距离末尾约 5 项时触发下一页； 结合加载中、hasMore、 分页暂停状态控制	说明分页数据请求与图片预加载是两个不同环节	不宣称任意网络下都能无缝衔接
再次浏览避免完全从头加载	Glide 图片缓存； 首页首屏列表磁盘快照； 按账号隔离；启动先展示快照， 再网络刷新	说明不同数据为什么需要不同缓存策略	首屏快照有效期 24 小时； 关注流不使用该磁盘快照； 快照不能保证作品仍然可访问
图片清晰且内存占用可控	Glide 内存缓存预算为最大堆的 1/3，磁盘 250MiB； 详情按屏幕尺寸解码；全屏图指定 2048×2048 目标尺寸； 内存压力时释放缓存	讲清图片尺寸、复用与资源预算的取舍	这是配置及目标解码尺寸， 不是整个进程内存硬上限， 不保证所有设备都无 OOM

用户需求	已找到的实现	汇报价值	边界
返回页面继续原来的位置	底部导航复用 Fragment；首页各 Tab 保存列表、游标和滚动状态；搜索使用 ViewModel 保存已加载状态	说明连续体验也依赖状态管理	页面恢复不等于进程被杀后完整 写入后的数据刷新另有规则
缓存不串账号、 不遮蔽状态变化	Feed 按账号隔离；空列表清理旧快照；请求 generation/session 校验；服务端详情缓存拆分静态与动态字段	说明缓存正确性与体验必须同时考虑	缓存不会消除所有陈旧窗口

来源：预加载有早期代码基础，近期工作新增真机 A/B 实验（`tools/analyze_preload.py` 聚合 `PF_ImgTiming` / `PF_Preload` 日志），把预加载价值与预解码取舍用数据固化；个性化推荐链路见 `docs/推荐技术与数据链路复盘-20261111.md`，首页接入方案见 `docs/android/推荐闭环实现方案-20261111.md`；缓存预算、快照和状态管理为既有基础。

主线 B：信息获取与整理

用户需求	已找到的实现	汇报价值	
提问后获取站内相关信息	专用智能搜索组合检索与综合回答；通用 Copilot 可由模型选择 `search_site` 工具；结果可带作品引用	讲清检索、生成与来源的关系	专用智能搜索是有界 通用工具阶段最多处 不是无限自主规划
结合正在看的内容完成任务	Java 按 detail/create/messages/general 构造上下文；Python 输出回复和结构化动作；Java 再校验动作范围	解释页面上下文如何转化为受约束的业务任务	不能只凭提示词宣称
专项能力能够重复使用	注册表有六项 Skill；统一输入输出模型、scope、身份、期限及错误码	展示业务能力的结构化组织方式	注册能力不代表每个 通用 Copilot 搜索走
生成期间继续浏览和互动	普通请求与 AI 请求分开调度；AI 执行池 2 并发、等待队列容量 4；已有流式结果展示	将并发调度与用户体验联系起来	普通请求仍串行； 不宣称全系统请求已
离开页面后不用重新生成	`AiTaskRegistry` 维护任务身份、状态、结果、订阅和 generation；切换账号清理	展示任务生命周期独立于页面生命周期	属于进程内注册表， 不把所有 Copilot 会
辅助创作兼顾效率与控制	单目标创作流式预览；完成后在原稿未变化时自动填入本地编辑内容，保留撤销；Copilot 使用待确认提案	展示按操作风险选择不同交互	两条创作路径必须分 “所有建议都手动采
外部智能客户端也能操作社区	MCP 提供 14 个工具，使用 OAuth 2.1 + PKCE，按用户身份和 scope 转发 Java 接口并审计	展示已有业务能力的复用	MCP 不直接操作数据 提交审核不等于公开 写操作都由服务端弹

三、正式 PPT 建议：11 页

页码	标题	在故事中的位置
1	浏览与创作中的两类需求	用用户行为定义整个汇报
2	图片的旅程：网络、磁盘与内存	连续浏览：图片从哪里来
3	预加载的价值（对照 ON/OFF）	连续浏览：提前网络
4	预解码的坑（要不要解码）	连续浏览：按需解码
5	行为画像：从行为到偏好	连续浏览：个性化从哪里来
6	规则推荐：从画像到个性化 Feed	连续浏览：个性化如何落地
7	AI 运行期间的交互连续性	连接浏览体验与智能任务
8	面向页面上下文的 Agent	信息处理：从用户意图得到业务提案

页码	标题	在故事中的位置
9	Skill 的契约与流程组合	信息处理：能力如何组织和复用
10	创作结果的应用与冲突处理	信息处理：怎样把结果安全地用于作品
11	MCP 对社区能力的复用	扩展到外部 Agent，衔接演示

十一页不是十一个独立专题：第 2–4 页讲图片预加载（旅程 → 价值 → 取舍），第 5–6 页讲个性化推荐（行为 → 画像 → 规则推荐），第 7 页完成转折，第 8–11 页展开信息处理。无需额外章节封面或“谢谢”页。

整体版式规范（给排版 Agent 的统一约定）

- **配色**：以深蓝为主色（技术感），配合两个语义色——红/橙表示“慢、坑、风险”，绿表示“快、好、已解决”。所有图例与箭头沿用同一配色，不额外引入多色。
- **图示风格**：统一用“圆角方块 + 实线箭头”的流程图；每张图节点不超过 5 个，节点内写名词、节点下写一句职责或数据；箭头方向从左到右或从上到下，不交叉。
- **数据呈现**：关键数字（如 50%→7.4%、13~23ms）用大号加粗数字单独成块，配一行小字说明；对比用左右两个卡片或上下两行表，不堆在一段文字里。
- **每页一个视觉焦点**：一页只突出一个主图（流程图/对照表/时间轴），辅助文字放主图下方，避免并排两个等权重的大图。
- **图表类型约定**：链路用横向流程图；对照用双卡片或两列表；时间线用横轴三点；公式用等宽字体的独立行；表格只保留“关键列 + 数字”，不放大表。
- **截图 vs 示意图**：来自项目真实界面的截图（瀑布流、搜索、创作页、详情等）一律标“【截图占位】”，Agent 只留一个带文字说明的空白占位框，**不自行生成图片**；其余流程图、对照表、时间轴、公式由 Agent 按本规范绘制。

第 1 页：浏览与创作中的两类需求

页面目的：让老师立即理解产品形态与本次技术重点。

上屏内容（版面）：

- **版式**：左右对称双栏，页底一条三端关系横条。
- **左栏**：【截图占位】双列瀑布流界面，顶部加蓝色标签“连续浏览”。
- **右栏**：【截图占位】搜索/创作界面，顶部加蓝色标签“获取与整理信息”。
- **页底横条**：三个圆角方块“Android/Web 客户端 → Java 业务服务 → Python 智能服务”，用实线箭头连接，标注数据/调用流向。
- **标题区**：顶部“PixelFlow：围绕浏览与创作的系统设计”+ 团队信息。

讲述要点：我们围绕两类使用行为设计系统：浏览时减少加载和状态中断，有明确目标时帮助用户查找资料、整理为作品。

下一页过渡：先看最频繁的操作，用户持续滑动瀑布流时，系统需要提前准备什么。

第 2 页：图片的旅程——网络、磁盘与内存

页面目的：讲清一张图从请求到显示经过的完整链路，以及为什么网络是最慢的一环。

上屏内容（版面）：

- **版式**：横向四节点流程图（从左到右）。
- **主图**：四个圆角方块用实线箭头连接——① 网络下载（红/橙，标“慢”）→ ② 落磁盘缓存（绿）→ ③ 解码进内存缓存（绿）→ ④ 显示（绿）。
- **数据标注**：每个方块下方写耗时——网络“250~1000ms（最慢 4.1s）”、磁盘解码“13$_{23ms”、内存“0}$1ms”；慢的红字、快的绿字。
- **一句结论**：图下方一行加粗小字“网络是最慢的一环，磁盘和内存都快”。

技术展开：

- 图片加载分四步：从网络下载原始文件、写入磁盘缓存、解码进内存缓存、交给控件显示。客户端用 Glide 缓存磁盘和内存。
- 图片由 Java 提供（demo 素材走 classpath，上传媒体存 MySQL `content_bytes` BLOB）；服务端用 Caffeine 缓存作品正文和图片字节。
- 真机测量：网络下载 250~1000ms（最慢 4.1s），磁盘解码 13~23ms，内存命中 0~1ms。**网络是最慢的一环，磁盘和内存都快。**
- Glide 的缓存命中条件是“URL + 尺寸 + 裁剪变换 + 类型”完全一致才复用；磁盘到内存需要一次“解码”。

讲述重点：网络最慢，这是后面“预加载 = 把网络提前”的动机。数据来自真机埋点，不是推断。

第 3 页：预加载的价值（对照 ON / OFF）

页面目的：用对照数据说明预加载为什么值得做。

上屏内容（版面）：

- **版式**：左右两个大数字卡片对照。
- **左卡片**：红框，标题“预加载 OFF”，大号数字“50.0%”，下一行小字“一半的图显示时才下载，大量占位”。
- **右卡片**：绿框，标题“预加载 ON”，大号数字“7.4%”，下一行小字“网络等待被提前，基本就绪”。
- **中间**：一个向下加粗箭头“50% → 7.4%”，强调降幅。
- **可配截图**：【截图占位】卡片下方并排放一张“占位空白”与一张“正常显示”的卡片截图对照。

技术展开：

- 关闭预加载时，一半的图是在用户滑到它时才从网络下载；开启后降到 7.4%。
- 预加载的本质：把“最慢的网络下载”提前到用户滑到之前完成，用户看到时图已就绪。
- 触发方式（滚动方向、相邻 3 项、500ms 节流、距离底部约 5 项分页）一句话带过，说明“怎么决定预加载什么”。

讲述重点：预加载的价值——网络占比 50% → 7.4%。

第 4 页：预解码的坑（要不要解码）

页面目的：讲清“只下载、不预解码”为什么不行，引出“预解码必须对齐显示参数”。

上屏内容（版面）：

- **版式**：三阶段横轴（假设 → 踩坑 → 修正），下方一张两行对照表。
- **三阶段横轴**：三个节点——① 假设“解码不耗时，只下载就行”（灰）→ ② 踩坑“加载失败变多 12.5%”（红）→ ③ 修正“对齐解码 7.4% + 内存命中 47.3%”（绿）。
- **对照表**：只下载不预解码 = 12.5%（红）+ 失败变多；预解码对齐 = 7.4%（绿）+ 失败消失。
- **一句话结论**：表下方加粗“必须预解码，且解码成显示真正要的尺寸”。

技术展开：

1. **一开始的假设**：网络是瓶颈，解码只要 13~23ms，那“只下载原始文件、不预解码”是不是就够了？
2. **踩的坑**：结果下载的文件缓存 key 对不上（尺寸/裁剪不一致），显示又走网络，加载失败反而变多（REMOTE 回升到 12.5%）。
3. **修正**：必须预解码，且解码成显示真正要的尺寸 + 裁剪，让缓存 key 对齐 → 显示直接命中内存（MEMORY 47.3%），失败消失。

讲述重点：**“测量 → 假设 → 验证 → 修正”** 的工程闭环，这是本故事的点睛之笔。

第 5 页：行为画像——从行为到偏好

页面目的：说明个性化不是黑盒，而是“先采集行为、再聚合成可解释偏好”。

上屏内容（版面）：

- **版式**：上半横向数据流图，下半一张四行权重表。
- **数据流图**：三个圆角方块用实线箭头连接——① [曝光/点击/停留/点赞] → ② [跨服务增量同步（Java→Python）] → ③ [画像：主题 / 作者 / 媒体偏好]。
- **权重表**：四行——点击 1 分（每日 1 次）、停留 1 分/30 秒（日上限 120s）、有效点赞 3 分（取消移除）、曝光 0 分（只统计）。
- **强调**：权重表里“点赞”一行加绿框，标注“以服务端业务结果为准”。

技术展开：

- **有边界**：不训练模型、不用大模型，只做最近 30 天的规则聚合，每条规则可逐字说明。
- **动作可校验**：点赞以服务端业务结果为准（不是客户端自报“我点了赞”），从源头避免刷兴趣分。
- **跨服务同步**：Java 存行为事件（递增 seq）→ Python 按游标增量拉取 → 事件写入、画像更新、游标推进在同一事务内原子完成。
- **画像三维度**：主题（标题关键词表识别，如“咖啡/面包”归入美食）、作者、媒体（图/视频）。主题不用语义向量、不解析图片内容——诚实的规则边界。

讲述重点：画像不是每次请求临时算的，而是行为先行、异步聚合，天然有更新延迟。

第 6 页：规则推荐——从画像到个性化 Feed

页面目的：讲清“画像 → 三个 Skill → 首页”的可解释推荐闭环。

上屏内容（版面）：

- **版式**：上半四个 Skill 方块链，下半打分公式 + 多样性约束两行。

- **Skill 链**：四个圆角方块——[profile_insight 判断画像可用] → [candidate_recall 召回] → [feed_rank 排序] → [首页个性化 Feed]，箭头连接，每个方块下写一句职责。
- **打分公式**：等宽字体独立行“推荐分数 = 4×主题匹配 + 2×作者偏好 + 媒体偏好 + 热度 + 新鲜度”。
- **多样性约束**：一行小字“最多 10 条、同一作者 ≤2、同一主题 ≤3”。
- **强调**：feed_rank 方块旁加绿字“每条结果带理由”，顶部加一句“不是黑盒大模型”。

技术展开：

- **经 Skill 抽象**：推荐是三个 Skill 的固定组合（不是 Agent、不是黑盒大模型）—— `profile_insight` 判断画像可用性 → `candidate_recall` 召回（兴趣优先 + 热门补充）→ `feed_rank` 排序。
- **有边界**：候选池最多 50 条、返回最多 10 条、同一作者 ≤2、同一主题 ≤3；不满足就少返回。
- **可解释**：每条结果带分项得分和规则理由，能回答“为什么排在这里”。
- **可校验**：交付前快照复核——作品撤回、偏好版本变化就丢弃结果，不交付旧快照。
- **降级 + 人控制**：画像不足/过期/冷启动自动降级 `popular` ；用户可屏蔽作者、抑制主题、关闭个性化，反馈分两条路径生效。

讲述重点：相关性之外还要控制作者/主题集中度；推荐是“有界、可解释、可降级”的确定性编排——和第 9 页的 Skill 契约是同一种工程纪律。

第 7 页：AI 运行期间的交互连续性

页面目的：自然引出第二条业务主线，说明智能任务带来的客户端问题。

上屏内容（版面）：

- **版式**：上半用户时间线，下半左右两个机制图。
- **时间线**：三个节点横排——[发起智能搜索] → [继续浏览] → [返回查看结果]，箭头连接，标出“AI 运行期间仍在用社区”。
- **左机制图**：两个并列队列方块——[普通业务 单线程串行队列]（绿）+ [AI 任务 2 线程池 + 4 队列]（蓝），下方小字“30s 等待超时、满则拒绝”。
- **右机制图**：一个[任务注册表]方块 + 一个[页面]方块，双向箭头标注“按身份订阅状态”。
- **配截图**：【截图占位】普通列表仍可操作 + AI 生成状态（体现“AI 运行期间仍能继续浏览”）。

技术展开：

- `CommunityClient` 保留普通业务**单线程串行队列**（保证写操作顺序与 token 轮换一致）；耗时 AI 走独立 **2 线程池 + 4 容量有界队列**，避免在同一个串行队列里等模型。
- AI 队列的背压分三步：2 并发 + 队列 4；**等待超过 30 秒**或**队列满**时立即失败提示，不无限堆积。超时也分档：普通请求 20 秒，AI 聚合接口（搜索/提炼/创作辅助）放宽到 55 秒。
- `AiTaskRegistry` 通过服务地址、账号、会话版本、场景和输入标识任务；页面重建后复用，避免自动重复发起。任务状态是显式状态机（NEW→WAITING→PREPARING→GENERATING→完成/取消/失败/中断/过期）。
- 搜索、提炼等已有增量展示；显示了部分内容不等于任务已完成。切换账号取消并清理旧任务。

讲述重点：任务生命周期与页面生命周期分离；“分队列 + 有界 + 超时”三步共同防止 AI 阻塞普通浏览。

限定：这些机制减少应用内的排队与重复请求，不构成模型生成更快的性能结论，也不提供杀进程后的后台续跑。

第 8 页：面向页面上下文的 Agent

页面目的：让老师看到 Agent 的具体输入、工具与业务输出。

上屏内容（版面）：

- **版式**：横向四步闭环流程图（从左到右），下方一行其他场景标注。
- **主图**：四个圆角方块——① [页面上下文（Java 构造 detail/create/messages/general）] → ② [模型判断 + 受限工具 search_site] → ③ [结构化动作 CREATE_DRAFT 提案] → ④ [Java 校验白名单 → 客户端展示待处理提案]，箭头连接。
- **强调**：第④方块加绿框，标“系统校验后才执行”。
- **其他场景**：图下方一行小字“详情→评论提案、草稿→修改提案、消息→指定会话回复”。
- **配截图**：【截图占位】通用对话的搜索引用 + 草稿提案（体现“模型检索 + 提案展示”）。

技术展开：

- 已绑定作品/草稿/消息的场景，由 Java 读取并校验业务上下文；模型不能通过自报字段获取任意用户数据。
- 通用对话允许模型决定是否调用站内检索工具；当前工具阶段有次数和结果范围限制。
- 动作按场景白名单校验，Python 清理一次，Java 再检查类型、长度和目标范围。
- 模型返回的是可供用户处理的业务提案，不是直接操作数据库。

必须区分：专用智能搜索接口采用单轮检索后综合回答；通用 Copilot 的模型工具选择是另一条路径。不要将两者包装成统一的无限自主规划系统。

讲述重点：Agent 不是黑盒，而是“有界输入 + 白名单动作 + Java 复核”的受约束流程——模型只提提案，系统校验后才执行。

第 9 页：Skill 的契约与流程组合

页面目的：讲清 Skill 为什么值得独立设计，以及它实际在哪些流程中使用。

上屏内容（版面）：

- **版式**：中央一个“契约”卡片，左右各一条组合链。
- **中央卡片**：圆角方块，标题“Skill 统一契约”，内列四行——名称、版本、输入输出模型、所需 scope。
- **左组合链**：[profile_insight] → [candidate_recall] → [feed_rank]，标注“推荐预览”。
- **右组合链**：[creator_assist] → [content_safety]，标注“非流式辅助创作”。
- **角落标注**：一个灰色小方块 [search_posts]，标注“专用智能搜索”。

技术展开：

- 注册表定义名称、版本、输入输出模型和所需 scope。
- 调用统一校验可信身份、参数模型、输出归属及协作式截止时间，返回 trace 标识、版本和耗时。
- `creator_assist` 可以注入模型处理器； `content_safety` 当前是规则检查，不宣称有完整内容理解或平台级安全能力。
- 能力可以按业务流程组合，但具体流程仍由调用方组织。固定组合不等同于模型自主选工具。

重要边界：通用 Copilot 的 `search_site` 不经过这个 Skill 注册表；流式创作直接调用模型流，再调用安全 Skill。页面应标注示例流程，不能画成“所有 Agent 都统一经过全部 Skill”。

讲述重点：可复用的业务能力与统一执行约束——能力被抽象成契约，Agent 才能安全复用，而不是每次直接拼数据。六个名字不需要逐个念。

第 10 页：创作结果的应用与冲突处理

页面目的：用明确的设计取舍讲清“提高创作效率”与“保留用户控制”如何共存。

上屏内容（版面）：

- **版式**：上下两个路径卡片对照，各标“风险等级”。
- **上卡片（低风险，绿框）**：“创作页 AI 辅助”——单目标流式预览 → 自动填入未修改的本地草稿 → 可撤销；右侧标“低风险：自动 + 撤销”。
- **下卡片（高风险，橙框）**：“Copilot 草稿提案”——用户确认 → 调用草稿应用接口 → 带 expected_version，冲突返回 409 重新生成；右侧标“高风险：提案 + 确认 + 版本校验”。
- **一句话结论**：两卡片下方加粗“效率来自自动，安全来自人保留最终控制”。
- **配截图**：【截图占位】创作页流式结果 + 撤销入口 + Copilot 提案确认界面。

技术展开：

- 自动填入编辑器与提交发布不同。用户仍在创作流程中管理内容，公开发布需要后续提交审核。
- 创作页记录生成时的文本快照与采纳前后状态，避免迟到结果覆盖用户新编辑，并支持撤销。
- Copilot 面向已保存草稿使用服务端内容版本，校验失败时保留冲突语义，不用“最后返回的结果覆盖一切”。
- 流式文本只是过程展示；结构化完成结果到达后才进入相应的应用逻辑。

讲述重点：按风险分级——低风险（本地草稿）自动填入 + 可撤销；高风险（已保存草稿）提案 + 确认 + 版本校验。效率来自自动，安全来自人保留最终控制。

第 11 页：MCP 对社区能力的复用

页面目的：解释相同社区能力怎样服务外部 Agent，并为演示收尾。

上屏内容（版面）：

- **版式**：纵向三层图（外部 Agent → MCP 适配层 → Java 业务服务）。
- **顶层**：一个[外部 Agent]方块。
- **中层（重点）**：一个[MCP 适配层]方块，内列三行——OAuth 2.1 + PKCE、按 scope 校验、工具调用审计。
- **底层**：[Java 业务服务]方块，标注“维护作品状态、权限、审核”。
- **工具分组**：中层下方三组小字——读取检索 / 草稿管理 / 互动，举三个例子（搜索、创建草稿、提交审核）。

技术展开：

- OAuth 2.1 + PKCE 绑定用户，工具按 scope 校验，转发既有业务接口。
- Java 继续维护作品状态、权限与审核规则；MCP 不直接写库。
- 调用记录工具、状态和耗时，不把工具审计日志称为模型完整思维过程。

讲述重点：对内是 Skill/Agent，对外是 MCP 工具，同一套“有边界、可校验”的复用。

结尾句：接下来通过连续浏览和信息整理两个场景，展示这些设计在应用中的实际表现。

限定：应用内 Agent 不要求绕行 MCP。MCP 的服务端鉴权与业务约束，也不等于每次写操作都自动获得人类逐项确认。

四、素材准备与排版原则

截图素材（后续人工填入）

各页标注“【截图占位】”的位置，后续从项目真实界面截图填入。候选素材位于 `docs/testing/copilot/` 和 `docs/testing/issue27/`（历史截图，使用前需核对与最终演示界面一致）。

讲述与排版原则

- 每页先讲用户遇到的一个具体问题，再讲机制和关键取舍。
- 代码仅保留必要的字段或调用名，不整屏粘源码。
- 页面可标少量参数，例如“相邻 3 项”“内容版本”；其余参数放备注。
- 试讲超时，优先删缓存容量、状态枚举和工具清单，保留两条主线与实际调用关系。
- 不另加架构概览、全功能列表或测试数字页挤占时间。